

QUESTION-15

Splay Tree

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 struct Node {
4     int key;
5     struct Node *left, *right;
6 };
7 struct Node* newNode(int key) {
8     struct Node* node = (struct Node*)malloc(sizeof(struct Node));
9     node->key = key;
10    node->left = node->right = NULL;
11    return node;
12 }
13 struct Node* rightRotate(struct Node* x) {
14     struct Node* y = x->left;
15     x->left = y->right;
16     y->right = x;
17     return y;
18 }
19 struct Node* leftRotate(struct Node* x) {
20     struct Node* y = x->right;
21     x->right = y->left;
22     y->left = x;
23     return y;
24 }
25 struct Node* splay(struct Node* root, int key) {
26     if (root == NULL || root->key == key)
27         return root;
28     if (key < root->key) {
29         if (root->left == NULL)
```

```
Preorder traversal before search:
30 40 50 100 200
Preorder traversal after searching 50:
50 40 30 100 200
```

=== Code Execution Successful ===

```
30         return root;
31     if (key < root->left->key) {
32         root->left->left = splay(root->left->left, key);
33         root = rightRotate(root);
34     }
35     else if (key > root->left->key) {
36         root->left->right = splay(root->left->right, key);
37         if (root->left->right != NULL)
38             root->left = leftRotate(root->left);
39     }
40     return (root->left == NULL) ? root : rightRotate(root);
41 }
42 else {
43     if (root->right == NULL)
44         return root;
45     if (key > root->right->key) {
46         root->right->right = splay(root->right->right, key);
47         root = leftRotate(root);
48     }
49     else if (key < root->right->key) {
50         root->right->left = splay(root->right->left, key);
51         if (root->right->left != NULL)
52             root->right = rightRotate(root->right);
53     }
54     return (root->right == NULL) ? root : leftRotate(root);
55 }
56 }
57 struct Node* insert(struct Node* root, int k) {
58     if (root == NULL)
```

```
Preorder traversal before search:
30 40 50 100 200
Preorder traversal after searching 50:
50 40 30 100 200
```

=== Code Execution Successful ===

<pre> 60 root = splay(root, k); 61 if (root->key == k) 62 return root; 63 struct Node* newnode = newNode(k); 64 if (k < root->key) { 65 newnode->right = root; 66 newnode->left = root->left; 67 root->left = NULL; 68 } 69 else { 70 newnode->left = root; 71 newnode->right = root->right; 72 root->right = NULL; 73 } 74 return newnode; 75 } 76 void preorder(struct Node* root) { 77 if (root != NULL) { 78 printf("%d ", root->key); 79 preorder(root->left); 80 preorder(root->right); 81 } 82 } 83 int main() { 84 struct Node* root = NULL; root = insert(root, 100); root = insert(root, 85 50); root = insert(root, 200); root = insert(root, 40); root = insert 86 (root, 30); 85 printf("Preorder traversal before search:\n"); 86 preorder(root); </pre>	<pre> Preorder traversal before search: 30 40 50 100 200 Preorder traversal after searching 50: 50 40 30 100 200 === Code Execution Successful === </pre>
--	--

```

85     printf("Preorder traversal before search:\n");
86     preorder(root);
87     root = splay(root, 50);
88     printf("\nPreorder traversal after searching 50:\n");
89     preorder(root);
90     return 0;
91 }

```